NIMBUS

Telemetry-Aware Adaptive Workload Scheduling on AWS

Can a scheduler that reads live worker telemetry place jobs better than one that reasons only about declared and reserved capacity?

REVIEW 1

Phase 3 of 9 complete · 274 tests passing

Presented by 〈 **Name** 〉 · Reg. No. 〈 **Number** 〉
Guide 〈 **Guide Name** 〉 · Department of 〈 ··· 〉

PROBLEM STATEMENT

Schedulers place work using numbers nobody measured

A cluster scheduler must answer one question for every job: which machine should run this?

Production placement engines answer it from paperwork rather than from measurement:

- **Declared requirements —** what the job's author claimed it would need
- **Reserved capacity —** the arithmetic sum of those claims on each host
- **Static attributes —** instance type, AZ, labels — fixed at registration

None of these is the machine's actual condition.

A host can sit at 100% CPU from work the scheduler never dispatched — a co-tenant, a backup job, an operator's build. Nothing was reserved, so it still advertises full capacity as free.

THE BLIND SPOT, CONCRETELY

Amazon ECS

Its cluster query language has no subject for observed CPU or memory utilisation. Placement filters and strategies read declared and reserved values only.

AWS Batch

Fair-share scheduling reasons about consumed queue share per tenant — not about host pressure.

The gap. Does adding observed, high-frequency, per-host telemetry to the placement function measurably improve scheduling — or does reservation accounting already capture everything that matters?

PROBLEM STATEMENT

Scoping: what this project does not claim

AWS already solves production job scheduling. Nimbus is not a replacement for any of it, and the project is forbidden from claiming otherwise.

ALREADY SOLVED BY AWS

- FIFO queues, job priority, dependsOn DAGs
- Fair-share scheduling with usage decay
- binpack / spread / random placement
- Lease semantics, retries, DLQs
- Target-tracking and predictive autoscaling
- Capacity-aware instance allocation

DELIBERATELY OUT OF SCOPE

- Multi-tenant fairness — starvation control only
- Autoscaling — the fleet is fixed at 3 workers
- Instance-type selection
- DAG dependencies between jobs
- Priority ordering — designed, not implemented
- Production security posture

THE NARROW THING NIMBUS TESTS

- Observed per-host CPU and memory pressure
- Smoothed over a short sample window
- Plus a least-squares utilisation trend
- Added to the per-job placement score
- Measured against the static baseline
- Treated as a hypothesis that may be rejected

The real test is not "does Adaptive beat FIFO" — it should, almost arithmetically. It is "does Adaptive beat Static Resource-Aware".

OBJECTIVE

One hypothesis, three falsifiable sub-claims

H1. For heterogeneous workloads on workers with dynamically changing resource utilisation, placement decisions informed by real-time telemetry and workload characteristics may improve completion latency, utilisation and load balance compared with FIFO or static resource-aware scheduling.

ID	Sub-hypothesis	Primary metric	Null
H1a	Adaptive reduces tail completion latency vs. FIFO	p95 completion time	no difference
H1b	Adaptive improves load balance vs. FIFO	CV of worker busy-time	no difference
H1c	Adaptive beats Static Resource-Aware — live telemetry adds something over reservation accounting	p95 completion time	no difference

H1c is the real test. H1a and H1b compare against baselines that ignore resources entirely. If Adaptive beats FIFO but not Resource-Aware, the honest conclusion — resource awareness helps, live telemetry adds nothing measurable — is a result, not a failure.

PRE-REGISTERED

Weights, metrics, α = 0.05 and the ≥5% materiality bar fixed in writing before the code existed

CONTROLLED

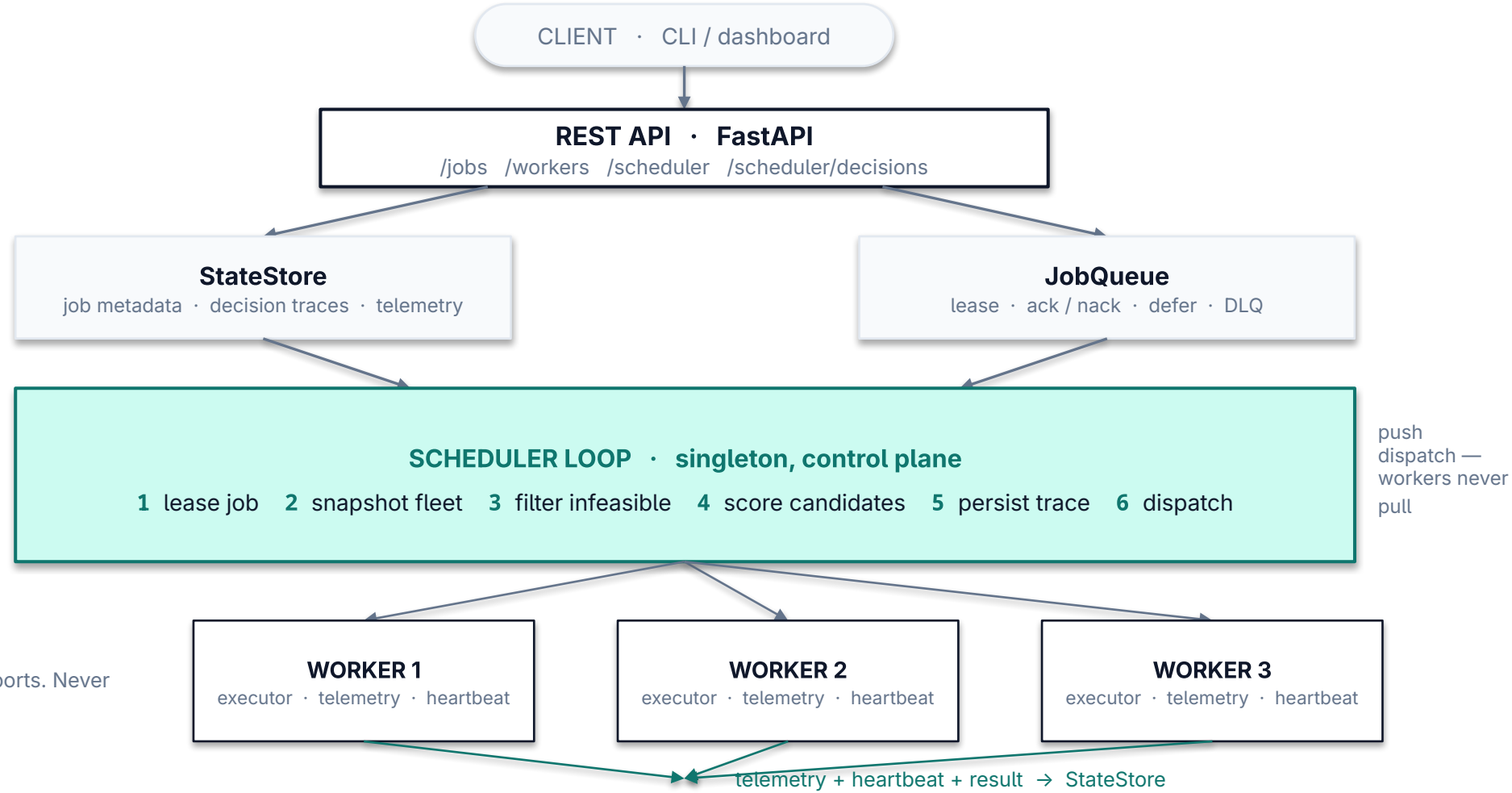
Every strategy uses FIFO ordering, so any difference is attributable to placement alone

REPLAYABLE

Every decision is a trace that can be re-scored counterfactually under another policy

SYSTEM ARCHITECTURE DESIGN

Control plane decides, data plane executes



DATA PLANE

Accepts, executes, reports. Never chooses what to run.

SYSTEM ARCHITECTURE DESIGN

The information boundary is the experiment

Worker state is split into three categories that are never conflated, because the central experiment depends on telling them apart. A placement policy never receives raw worker state — it declares its information budget through a view.

DECLARED

Operator configuration

vCPU, memory and slot capacity set at registration. Never changes at runtime.

→ **read by every policy**

RESERVED

Nimbus bookkeeping

Sum of what Nimbus has dispatched and not yet reclaimed. reserve() / release().

→ **read by every policy**

OBSERVED

psutil, sampled live

Smoothed CPU mean, memory pressure and least-squares trend over the last N samples.

→ **read by adaptive-v1 only**

Two views, pinned by tests, enforce that boundary:

`StaticWorkerView`	round_robin, static_resource_aware, static_resource_aware_binpack	sees: **DECLARED + RESERVED**
`AdaptiveWorkerView`	adaptive (adaptive-v1)	sees: **DECLARED + RESERVED + OBSERVED + telemetry quality**

Recorded ≠ consumed. Traces record all three categories for every candidate, written after the policy has returned — so replay can prove which fields a decision actually depended on.

SYSTEM ARCHITECTURE DESIGN

Ordering and placement are separate objects

Every strategy uses FIFO ordering, so a comparison isolates placement — the only variable is what the policy may see.

Strategy	Placement	Sees	Status
`fifo`	round-robin rotation	DECLARED + RESERVED	implemented
`static_resource_aware`	balance on projected utilisation	DECLARED + RESERVED	implemented
`static_resource_aware_binpack`	ECS-style binpack	DECLARED + RESERVED	implemented
adaptive	**adaptive-v1 weighted score**	**+ OBSERVED**	**implemented**
priority	*priority + aging*	—	*not implemented*

adaptive-v1 — score, frozen before the code was written

```
Score(w,j) = α · S_cpu          observed, smoothed
           + β · S_memory       observed
           + γ · S_reservation  the static model, kept
           + δ · S_queue        declared + reserved
           + ε · S_trend        least-squares slope
```

Weights vary by workload class

Class	α	β	γ	δ	ε
CPU_BOUND	**0.50**	0.10	0.20	0.15	0.05
MEMORY_BOUND	0.10	**0.50**	0.20	0.15	0.05
IO_BOUND	0.15	0.15	0.25	**0.40**	0.05
LIGHTWEIGHT	0.10	0.05	0.25	**0.55**	0.05

Telemetry ranks; it never changes eligibility. Feasibility is evaluated on static fields alone, stale telemetry falls back to reservation estimates rather than to "0% CPU", and there is no threshold cliff in the score.

REFERENCES

Sources and prior art

AWS SERVICE DOCUMENTATION — PRIOR ART

[1] Amazon Web Services, “AWS Batch job scheduling,” AWS Batch User Guide.
docs.aws.amazon.com/batch/latest/userguide/job_scheduling.html

[2] Amazon Web Services, “Fair-share scheduling policies,” AWS Batch User Guide.
docs.aws.amazon.com/batch/latest/userguide/fair-share-scheduling.html

[3] Amazon Web Services, “Compute environment allocation strategies,” AWS Batch User Guide.
docs.aws.amazon.com/batch/latest/userguide/allocation-strategies.html

[4] Amazon Web Services, “Amazon ECS task placement,” Amazon ECS Developer Guide.
docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement.html

[5] Amazon Web Services, “Amazon ECS task placement strategies,” Amazon ECS Developer Guide.
docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement-strategies.html

[6] Amazon Web Services, “Cluster query language,” Amazon ECS Developer Guide.
docs.aws.amazon.com/AmazonECS/latest/developerguide/cluster-query-language.html

[7] Amazon Web Services, “Amazon SQS visibility timeout,” Amazon SQS Developer Guide.
docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html

[8] Amazon Web Services, “Amazon SQS dead-letter queues,” Amazon SQS Developer Guide.
docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

[9] Amazon Web Services, “Amazon SQS fair queues,” Amazon SQS Developer Guide.
docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-fair-queues.html

[10] Amazon Web Services, “Scale your Auto Scaling group,” Amazon EC2 Auto Scaling User Guide.
docs.aws.amazon.com/autoscaling/ec2/userguide/scale-your-group.html

[11] Amazon Web Services, “AWS Price List Query API.”
pricing.us-east-1.amazonaws.com/offers/v1.0/aws/

TOOLS AND LIBRARIES

FastAPI · Uvicorn · Pydantic · psutil · NumPy · SciPy · pytest

PROJECT DESIGN DOCUMENTS

`architecture` · `scheduling` · `experiments` · `aws-native-scheduling` · `adaptive-v1-preregistration` · `limitations` (all under `docs/`)